

Documento de Sustentación: Taller CampusTasks

Universidad del Valle — Ingeniería de Sistemas
Desarrollo de Software II

1. Datos

Nombre de los integrantes	Samuel Alvarado Acevedo, Santiago Arboleda, Samuel Romero, Sebastián Sáenz
Código(s)	202459469 – 202459463 – 202459464 - 202459528
Fecha	06 / 10 / 2026
Nombre de la rama	taller/actualizar-eliminar-tareas-SamuelAlvarado

2. Puesta en marcha

Instalación de PostgreSQL

Para instalar PostgreSQL, se utilizó el instalador del sistema operativo para Windows. Se configuró con el usuario por defecto `postgres` y su respectiva contraseña.

Creación de la base de datos y la tabla

Usando `psql`, nos conectamos al servidor y ejecutamos los siguientes comandos para crear la base de datos y la tabla inicial:

```
CREATE DATABASE campus_tasks;  
\c campus_tasks  
CREATE TABLE tareas (id SERIAL PRIMARY KEY, titulo TEXT NOT NULL);  
INSERT INTO tareas (titulo) VALUES  
('Leer la guía de la clase 2'),  
('Preparar el entorno de desarrollo');
```

Configuración del archivo .env

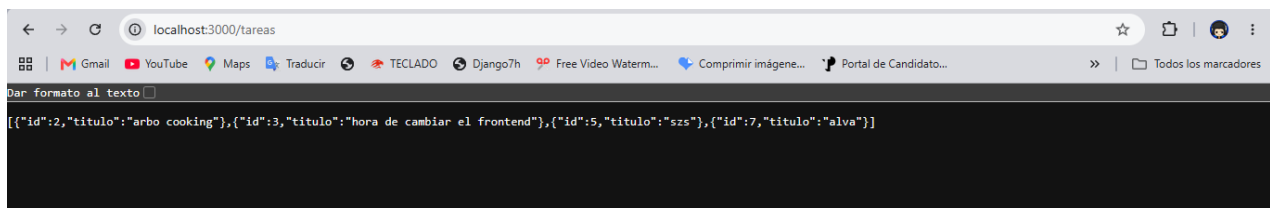
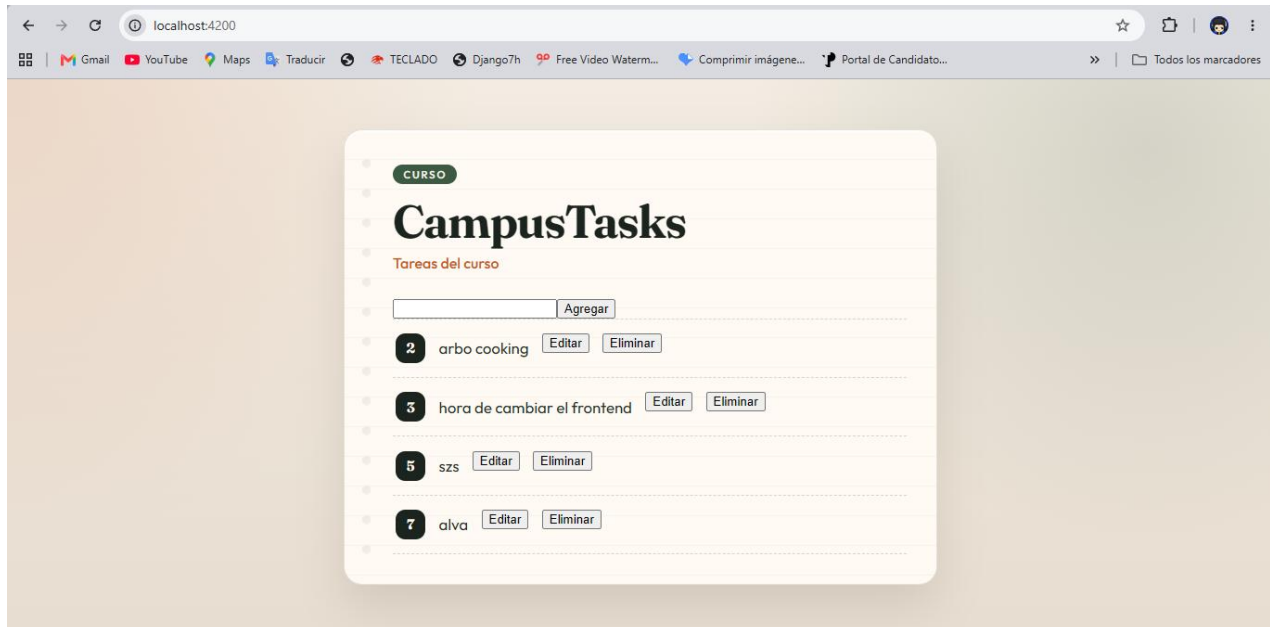
En el backend, se creó un archivo `backend/.env` con las variables requeridas por `DatabaseService` (omitiendo la contraseña por seguridad):

```
DB_HOST=127.0.0.1  
DB_PORT=5432  
DB_USER=postgres  
DB_PASSWORD=<omitido_por_seguridad>  
DB_NAME=campus_tasks
```

Comprobación del Backend y Frontend

Para el backend se ejecutó `npm ci` y `npm run start:dev`, comprobando que en `http://localhost:3000/tareas` devolviera el JSON correcto.

Para el frontend se ejecutó `npm ci` y `npm start`, comprobando que las tareas cargaran correctamente en la lista desde el navegador en <http://localhost:4200/>.



3. Lectura de los ejemplos

Al analizar los archivos de prueba existentes, entendimos lo siguiente:

- `tareas.service.spec.ts` (**Pruebas Unitarias**): Entendimos que la prueba unitaria no depende de una base de datos real. Utiliza un `mock` (un objeto falso o simulado llamado `query = jest.fn()`) inyectado en vez del `DatabaseService` real. En cada prueba se especifica qué debe retornar ese `mock` (`query.mockResolvedValue(...)`) y luego verificamos que se haya llamado con los parámetros y la consulta SQL correctos.
- `tareas.integration.spec.ts` (**Pruebas de Integración**): En estas pruebas, se levanta la aplicación `NestJS` de forma programática con el módulo simulado, simulando el servicio de la base de datos con

`jest.fn()`. Usamos `supertest` para realizar llamadas HTTP (GET, POST, PATCH, DELETE) a nuestras rutas, permitiendo probar cómo los controladores y servicios interactúan entre sí y si los status codes (200, 201, 404, etc.) devueltos son los correctos.

- `tareas.component.spec.ts` (**Pruebas de Componente Frontend**): Las pruebas utilizan Karma y Jasmine. En este caso se hace uso de un `spy` sobre el servicio `TareasService`. Así evitamos realizar llamadas HTTP reales al backend, simulando lo que el servicio debe devolver mediante `of(...)` de RxJS, y evaluando cómo responde el DOM del componente (por ejemplo, si aparece la tarea nueva en la lista).

Patrón reutilizado (Preparar, Ejecutar, Verificar)

Reutilizamos el patrón de 3 pasos:

1. **Preparar:** Definimos los datos simulados y qué debe responder el `mock` o `spy`.
2. **Ejecutar:** Hacemos la llamada al método del servicio (unitaria), hacemos el request con `supertest` (integración) o manipulamos el DOM emitiendo clicks (frontend).
3. **Verificar:** Usamos `expect` para comprobar la respuesta, estado y que la llamada (`spy` o `mock`) se haya hecho con los argumentos precisos.

4. Implementación

Backend

Métodos y rutas agregados

Operación	Servicio (<code>tareas.service.ts</code>)	Controlador (<code>tareas.controller.ts</code>)	SQL usado
Actualizar (Editar)	<code>actualizar(id: number, titulo: string):</code> actualiza la tarea y devuelve la fila correspondiente.	<code>Ruta @Patch('/:id'):</code> recibe el id en la URL y el nuevo título en el <code>@Body()</code> .	<code>UPDATE tareas SET titulo = (\$1) WHERE id = (\$2) RETURNING id, titulo</code>
Eliminar	<code>eliminar(id: number):</code> borra la tarea.	<code>Ruta @Delete('/:id'):</code> recibe el id.	<code>DELETE FROM tareas WHERE id = (\$1) RETURNING id, titulo</code>

Ambas consultas son parametrizadas (`$1`, `$2`), igual que `crear`, sin concatenar valores dentro del texto SQL.

Conversión del `:id` a `number`

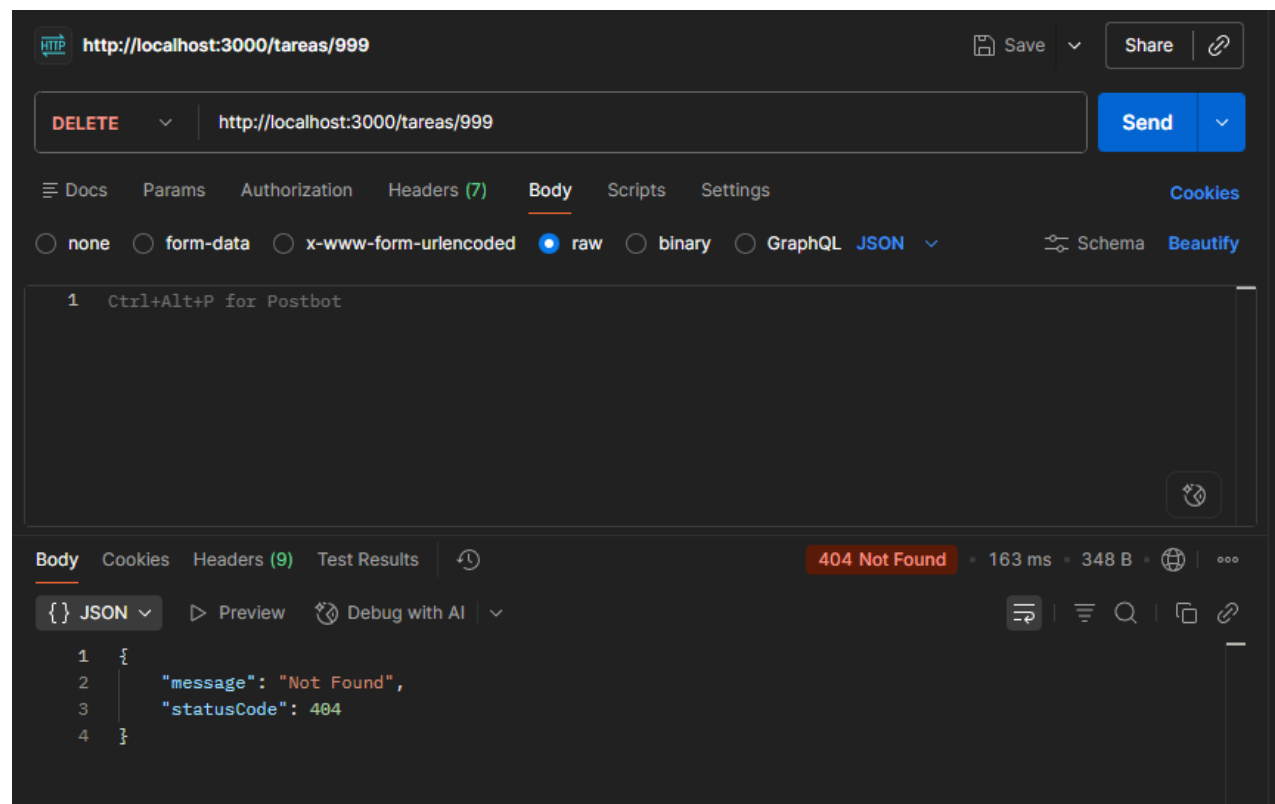
En el controlador decidimos que cuando le llega un `@param` como parte de la petición http en la url el toma ese id que le pasan como string y lo convertimos a int o number que en este caso es lo que necesitamos para poder llamar la función del servicio que hace el cambio en la base de datos, usamos la función `parseInt(idurl,10)` esta función recibe dos parámetros primero el string que queremos cambiar y segundo la base que estamos trabajando la cual es base 10.

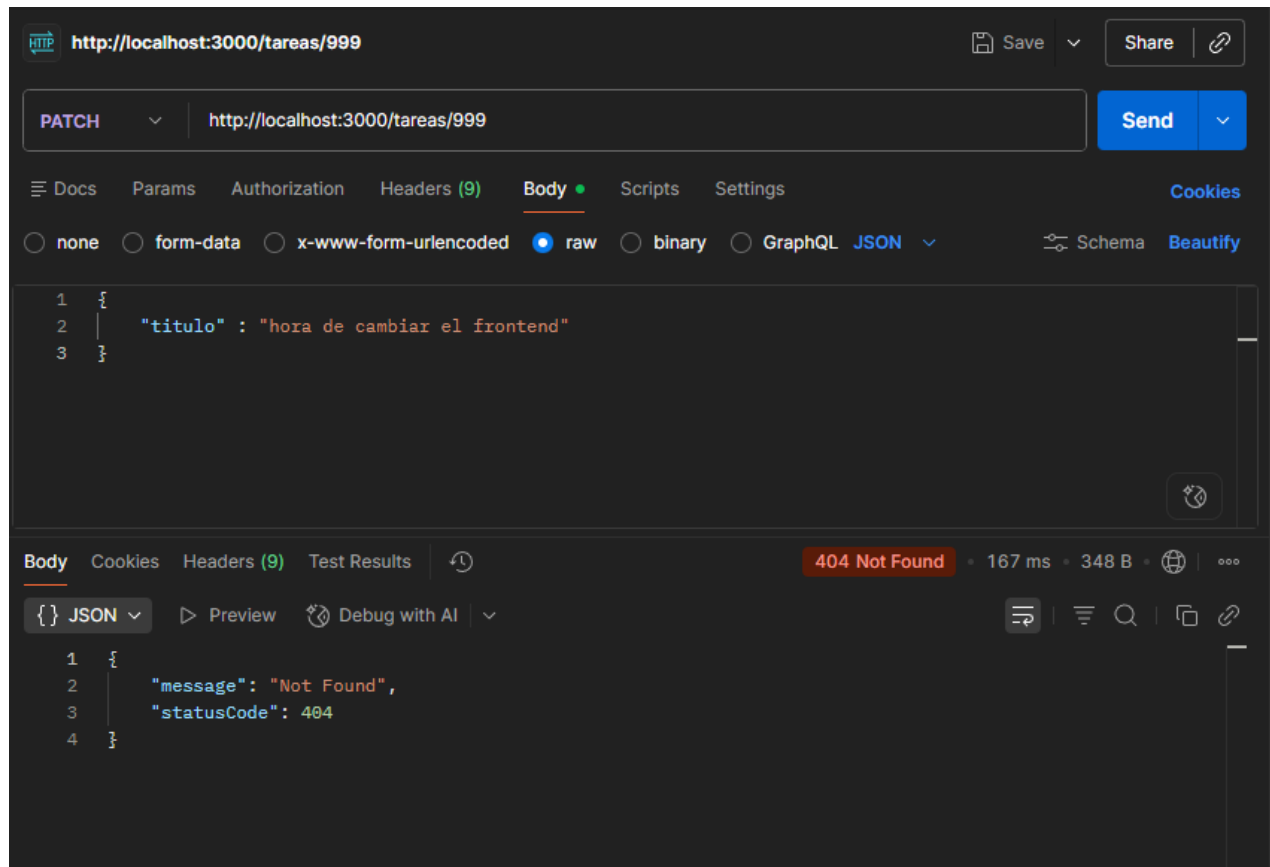
Decisión del error 404

En el controlador (`tareas.controller.ts`), decidimos que el 404 se emita analizando la respuesta del servicio. Dado que `query` siempre retorna un arreglo (que puede estar vacío), sabemos que si `resultado.rows[0]` es indefinido (`undefined`), la tarea no existía.

En los métodos de controlador (`actualizarTarea` y `eliminarTarea`), si la respuesta devuelta por el servicio es nula o indefinida, arrojamos la excepción de NestJS:

```
if (!TareaActualizada) { throw  
  new NotFoundException(); }
```





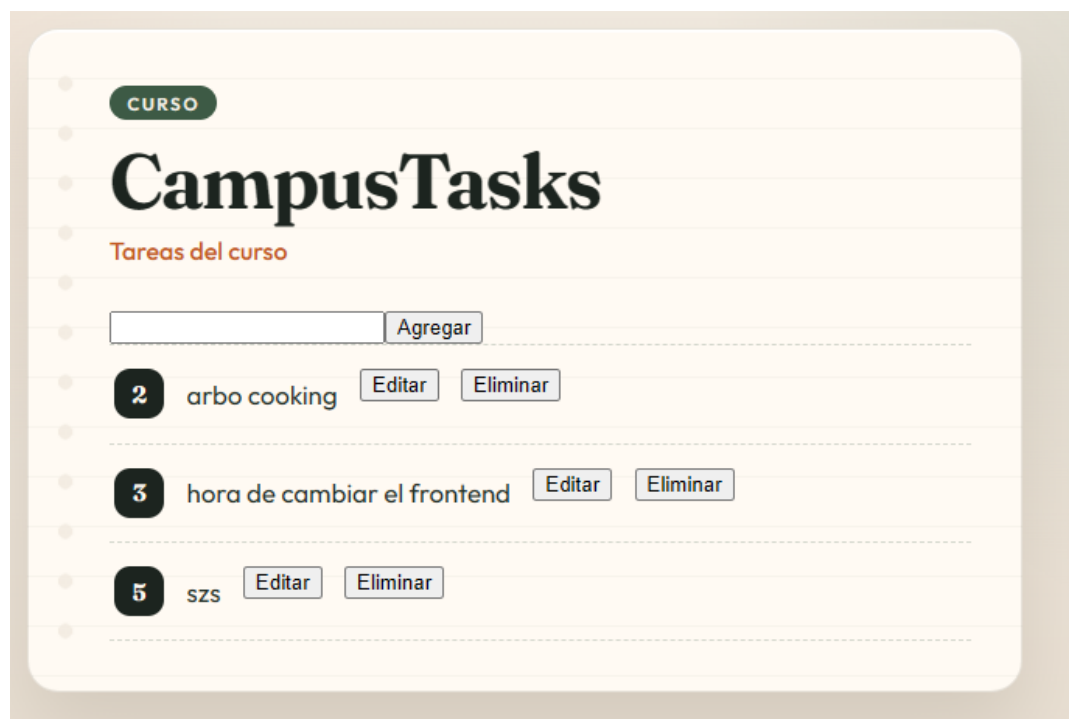
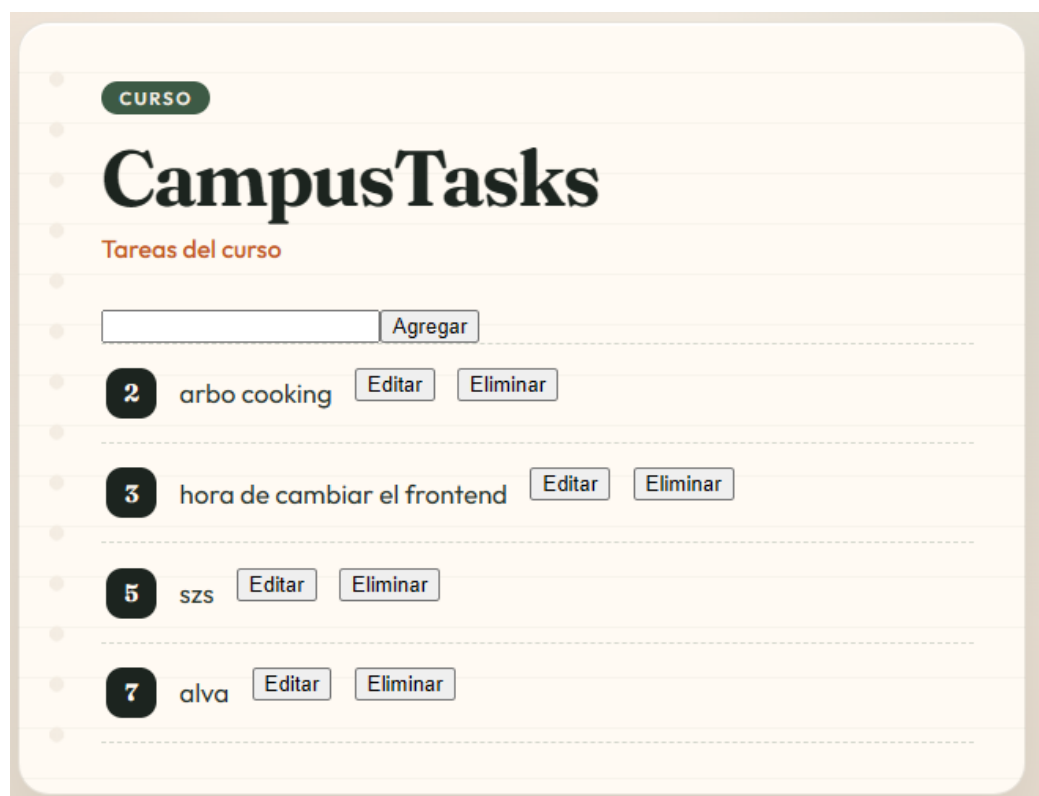
Frontend

Métodos del servicio HTTP (`tareas.service.ts`): Se agregaron `actualizar(id: number, titulo: string)` enviando un `PATCH`, y `eliminar(id: number)` enviando un `DELETE`. Ambos retornan `Observable<Tarea>`.

Botones y componentes (`tareas.component.ts` / `tareas.component.html`): En el HTML, la lista de tareas fue separada en dos modos basados en un `signal` llamado `editandoId`:

- **Modo normal:** Muestra el título y los botones **Editar** y **Eliminar**.
- **Modo edición:** Muestra un `<input>` con el título actual y los botones **Guardar** y **Cancelar**.

Qué pasa en pantalla ante un error: La pantalla de Angular sólo se actualiza **dentro del** `subscribe` cuando la respuesta es exitosa. Si el backend responde con un `404 Not Found` u otro código de error, el observable emitirá un error y no pasará a la ejecución exitosa de `subscribe`, lo que significa que **la tarea en pantalla no será editada ni removida de la vista**, quedando consistente en base a que en el backend tampoco pudo completarse la acción.



5. Pruebas

Se añadieron pruebas tanto en backend como en frontend usando el mismo patrón.

Tipo	Archivo	Prueba	Qué cubre
Unitaria (backend)	<code>tareas.service.spec.ts</code>	Actualiza el título y devuelve la fila creada	Cubre el UPDATE validando que se llame con el string y el id.
Unitaria (backend)	<code>tareas.service.spec.ts</code>	Elimina y devuelve la fila eliminada	Cubre el DELETE validando que ejecute la query correctamente devolviendo la fila.
Integración (backend)	<code>tareas.integration.spec.ts</code>	PATCH / recibe id y nuevo título y lo actualiza	Caso 200.
Integración (backend)	<code>tareas.integration.spec.ts</code>	PATCH / recibe id y nuevo título, error 404 porque el id no existe	Caso 404.
Integración (backend)	<code>tareas.integration.spec.ts</code>	DELETE / recibe id y elimina	Caso 200.
Integración (backend)	<code>tareas.integration.spec.ts</code>	DELETE / recibe id y elimina, error 404 porque el id no existe	Caso 404.
Componente (frontend)	<code>tareas.component.spec.ts</code>	debería actualizar la tarea al hacer clic en Editar y luego Guardar	Simula el clic en Editar, modifica el input, hace clic en Guardar y verifica que <code>tareasService.actualizar</code> haya sido llamado con los datos correctos y el DOM refleje el cambio.
Componente (frontend)	<code>tareas.component.spec.ts</code>	debería eliminar la tarea al hacer clic en Eliminar	Simula el clic en Eliminar y verifica que <code>tareasService.eliminar</code> haya sido llamado con el id correcto y que desaparezca del DOM.

Resultados de la ejecución

Tanto en la carpeta `backend` como `frontend`, al correr `npm test`, todas las suites de prueba pasaron exitosamente.

```

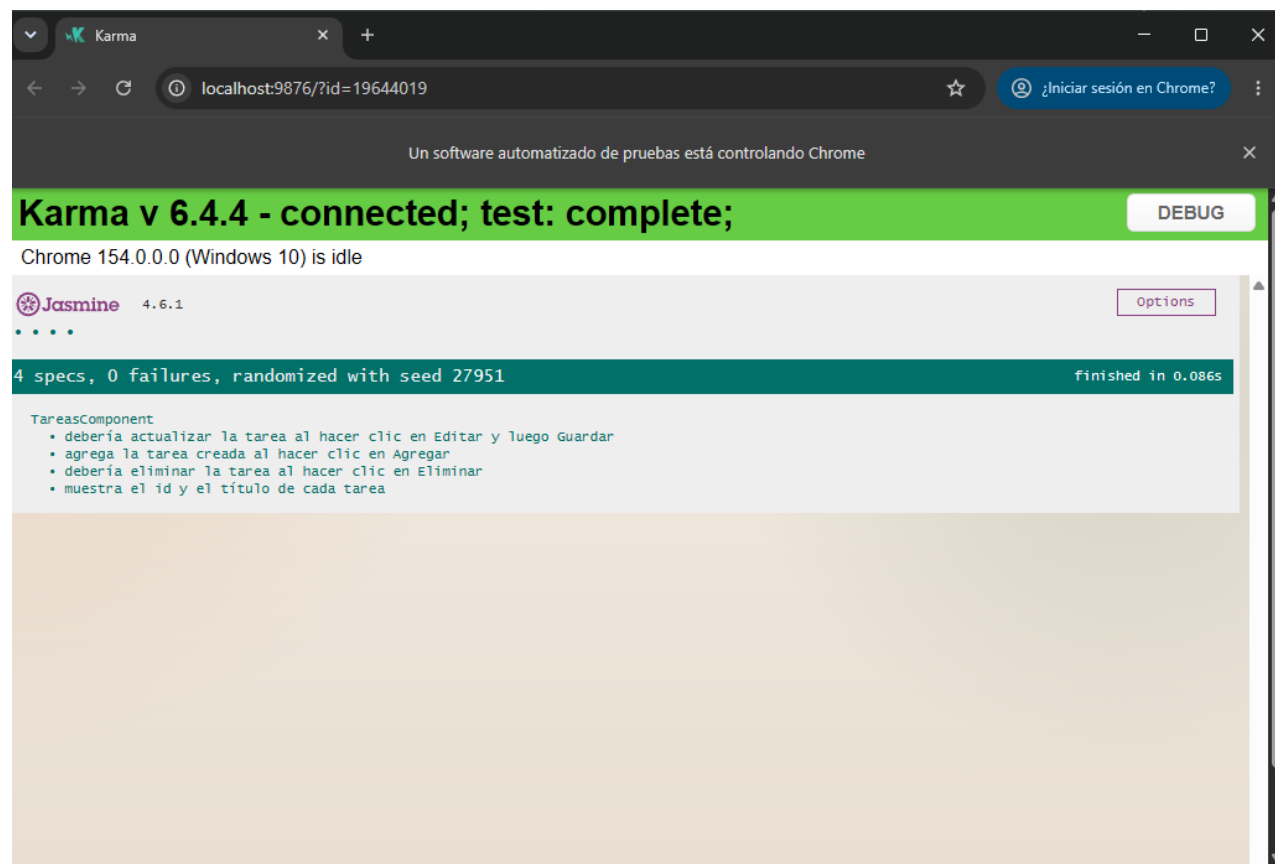
PS C:\Users\samue\OneDrive\Documentos\Sexto semestre\Desarrollo\
talleres\campus-task\backend> npm install
npm warn install-scripts
npm warn install-scripts Run `npm install-scripts ls` to review,
or `npm install-scripts approve <pkg>` to allow.
PS C:\Users\samue\OneDrive\Documentos\Sexto semestre\Desarrollo\
talleres\campus-task\backend> npm test

> backend@0.0.1 test
> jest

PASS src/tareas/tareas.service.spec.ts
PASS src/tareas/tareas.integration.spec.ts

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        2.5 s, estimated 14 s
Ran all test suites.
PS C:\Users\samue\OneDrive\Documentos\Sexto semestre\Desarrollo\
talleres\campus-task\backend>

```



6. Git

Para crear la rama, realizar los commits y enviarlo al servidor remoto, utilizamos los siguientes comandos:

```
# 1. Asegurarse de estar en la rama principal
git checkout master git pull origin master

# 2. Crear y movernos a la rama nueva git
checkout -b taller/actualizar-eliminar-tareas

# 3. Comprobación de estado (para evitar agregar el .env)
git status

# 4. Agregar los archivos y hacer el commit git add . git commit -m "Implementar
edición y eliminación de tareas en backend y frontend"

# 5. Subir la rama al remoto git push -u origin
taller/actualizar-eliminar-tareas
```

URL del repositorio/rama

<https://github.com/TevenV27/campus-task/tree/taller/actualizar-eliminar-SamuelAlvarado-SantiagoA-SamuelR-SebastianS>